



Examples of Time Series Filtering



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Filter Overview

Progressive Smoothing

No filter · Moving Average · EMA · Smoothing · LOWESS · Spline

Robust Filters

Winsor · QES

Frequency and Decomposition

FFT · Gabor · Wavelet · EMD · REMD

Trend and Seasonality

HP · Kalman · Seasonal Adjustment

Fifteen filters organized into four families, each targeting a specific type of unwanted variation in the time series.

Common Setup

```
# Load packages
library(daltoolbox)
library(tspredit)

# Series with noise and artificial spikes
data(tsd)
noise <- rnorm(length(tsd$y), 0, sd(tsd$y)/10)
spike <- rnorm(1, 0, sd(tsd$y))
tsd$y <- tsd$y + noise
tsd$y[10] <- tsd$y[10] + spike
tsd$y[20] <- tsd$y[20] + spike
tsd$y[30] <- tsd$y[30] + spike
```

- Load the `daltoolbox` and `tspredit` packages
- Load the `tsd` series with `data(tsd)`
- Generate Gaussian noise with `rnorm()` proportional to the series standard deviation
- Inject artificial spikes at positions 10, 20, and 30
- This setup is **shared by all filters** — the following slides show only the specific code for each filter

❏ **Note:** This preparation block is executed once before any filter. In the following slides, it is assumed that `tsd$y` is already prepared with noise and spikes.

Filter None – Identity

The identity filter returns the original series without any modification. Its role is to establish a reference baseline: all other filters should be compared against it to evaluate whether the applied transformation actually improves the prediction pipeline.

```
# Apply identity filter
filter <- ts_fil_none()
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- Instantiate the filter with `ts_fil_none()`
- Calibrate with `fit()` and transform with `transform()`
- The output series is identical to the input — no changes applied

❏ **Key concept:** The identity filter is the **baseline**. All other filters must be compared against it to measure the real denoising gain.

Moving Average Filter (MA)

The Moving Average filter replaces each point with the arithmetic mean of its neighbors within a fixed window. It is a fast and simple smoother, equivalent to a low-pass FIR filter in the time domain. Larger windows reduce variance more, but introduce lag and attenuate peaks — use it as a first denoising step for series with light noise.

```
# Apply MA filter with window = 3
filter <- ts_fil_ma(3)
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- Define the window size in the argument of `ts_fil_ma()`
- Each output point is the **arithmetic mean** of neighbors within the window
- Larger windows → more smoothing, but greater lag and peak attenuation
- Equivalent to a low-pass FIR filter in the time domain
- Use as a **first step** before more complex models

❏ **Key concept:** The `window` parameter controls the trade-off between variance (reduced) and bias (lag). Start with a small window (3–5).

EMA Filter – Exponential Moving Average

The EMA filter assigns exponentially decreasing weights to past observations, giving more importance to recent ones. This makes it more responsive to level changes than the simple Moving Average, while still reducing short-term noise.

```
# Apply EMA filter
filter <- ts_fil_ema(3)
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- Instantiate with `ts_fil_ema()` passing the window span
- Weights decay exponentially — recent observations have **more weight**
- Reacts faster to changes than simple MA
- Calibrate with `fit()` and apply with `transform()`
- Ideal when the series gradually changes level over time

📌 **Key concept:** EMA balances **denoising and recency**. Unlike MA, it does not treat all points in the window with equal weight. Ref.: Brown (1959).

Generic Smoothing Filter

The Generic Smoothing filter applies a standard smoothing operator to reduce short-term variations and reveal the underlying signal. It is a quick denoising option without needing to choose a specific family — if the result is over- or under-smoothed, migrate to LOWESS or Spline for finer control.

```
# Apply generic smoothing (light defaults)
```

```
filter <- ts_fil_smooth()
```

```
filter <- fit(filter, tsd$y)
```

```
y <- transform(filter, tsd$y)
```

```
plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- Use `ts_fil_smooth()` without arguments for default configuration
- Attenuates high-frequency components while preserving broad trends
- If the result is over- or under-smoothed, migrate to LOWESS or Spline
- Useful as a quick denoising step without choosing a specific family

📌 **Key concept:** General-purpose filter — start with the **defaults**. For fine control of the smoothness/fidelity trade-off, prefer Spline with `spar` or LOWESS with `f`. Ref.: Hastie et al. (2009).

LOWESS Filter – Local Regression

LOWESS (Locally Weighted Scatterplot Smoothing) fits local regressions along the series to estimate non-linear trends without imposing a global shape. The parameter `f` controls the fraction of the series used in each regression: smaller values produce more flexible curves; larger values produce smoother ones.

```
# Apply LOWESS filter (f = window fraction)
filter <- ts_fil_lowess(f = 0.2)
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- Parameter `f` defines the fraction of the series used in each local regression
- Smaller values of `f` → more flexible curve; larger values → smoother
- Fits local regressions along the series to estimate non-linear trends
- Assumes no global shape — ideal for trends that change behavior over time

📌 **Key concept:** LOWESS follows **flexible trends** without imposing a global shape. Unlike MA, it adapts locally. Ref.: Cleveland (1979).

Spline Filter

The Spline filter fits a smooth curve whose roughness is explicitly penalized via the `spar` parameter. Unlike LOWESS, the smoothness control is global and explicit: `spar` close to 0 keeps the curve close to the data; close to 1 produces a very smooth curve. Ideal when precise control of the fidelity/smoothness trade-off is desired.

```
# Apply Spline filter (spar controls smoothness)
filter <- ts_fil_spline(spar = 0.5)
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- Parameter `spar` controls the smoothness vs. data fidelity trade-off
- `spar` close to 0 → curve close to the data; close to 1 → very smooth
- Fits a curve whose roughness is explicitly penalized
- Unlike LOWESS: explicit and global control of smoothness

📌 **Key concept:** Spline offers **explicit control** of the fidelity/smoothness trade-off via `spar`. Ref.: Wahba (1990).

Winsor Filter – Outlier Control

The Winsor filter caps extreme values to less extreme thresholds, reducing the leverage of outliers without redrawing the entire signal. Unlike smoothers, it does not alter the overall shape of the series — it only protects downstream pipeline steps from isolated spikes.

```
# Apply Winsor filter
filter <- ts_fil_winsor()
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- Instantiate with `ts_fil_winsor()` — no mandatory parameters
- Extreme values are **capped** to less extreme thresholds
- Does not smooth the entire series — only reduces the leverage of outliers
- Protects downstream pipeline steps without redrawing the signal

📌 **Key concept:** Winsorization is a filter **focused on outliers**, not general smoothing. Use before models sensitive to spikes. Ref.: Wilcox (2012).

QES Filter – Quadratic Exponential Smoothing

QES (Quadratic Exponential Smoothing) generalizes simple and double exponential smoothing (Holt/Brown) by adding quadratic curvature components. This allows tracking series with accelerating or decelerating trends more precisely. The gamma parameter controls the adaptation of the trend gain.

```
# Apply QES (without gamma adaptation)
filter <- ts_fil_qes(gamma = FALSE)
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- QES generalizes simple and double smoothing (Holt/Brown) with curvature terms
- Parameter `gamma = FALSE` maintains default behavior without gain adaptation
- Models level and trend with exponentially weighted averages in a quadratic fashion
- Useful for trend-dominated series with gradual acceleration or deceleration

📌 **Key concept:** QES incorporates **higher-order trend components**, better tracking series with acceleration. Ref.: Gardner (1985, 2006).

FFT Filter – Frequency Domain

The FFT filter transforms the series into the frequency domain, suppresses high-frequency components (fast noise) via a low-pass filter, and reconstructs the signal via inverse FFT. It operates globally on the spectrum — unlike local smoothing — making it ideal when noise manifests as rapid oscillations, not isolated spikes.

```
# Apply FFT filter
filter <- ts_fil_fft()
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- The series is transformed into the **frequency domain** via FFT
- High-frequency components (fast noise) are suppressed
- The signal is reconstructed via inverse FFT
- Ideal when noise is understood as rapid oscillation, not isolated spikes

📌 **Key concept:** FFT connects denoising to **frequency analysis**. Unlike local smoothing — it operates globally on the spectrum. Ref.: Oppenheim & Schaffer (2010).

Gabor Filter – Localized Frequency

The Gabor filter combines a Gaussian envelope with a cosine carrier, creating a local convolution kernel that attenuates unwanted variations around a selected central frequency. It operates in the time domain with frequency awareness — unlike FFT (global) and Wavelet (multiscale), Gabor is localized and oriented to a specific band.

```
# Prepare sinusoidal series with noise
x <- seq(0, 6 * pi, length.out = 120)
y <- sin(x) + 0.3 * sin(8 * x) + rnorm(length(x), 0, 0.12)

# Apply Gabor filter
filter <- ts_fil_gabor(window_size = 3, central_frequency = 0.05)
filter <- daltoolbox::fit(filter, y)
yhat <- transform(filter, y)

plot_ts_pred(y = y, yadj = yhat)
```

Step by Step

- `window_size` controls the width of the Gaussian envelope in number of observations
- `central_frequency` in cycles per observation — must be between 0 and 0.5
- Values of `central_frequency` close to zero behave like Gaussian smoothing; larger values emphasize local oscillatory structure
- Uses a centered (symmetric) window — suitable for offline preprocessing, not for strict causal forecasting
- Reflection padding ensures the filtered series has the same length as the input

📌 **Key concept:** Gabor provides **frequency-localized smoothing** — unlike FFT (global spectrum) and Wavelet (multiscale). Ideal when the frequency of the noise to suppress is known. Ref.: Gabor (1946).

Wavelet Filter – Multiscale

The Wavelet filter decomposes the series into multiple resolution scales and removes the detail components associated with noise through thresholding. Unlike FFT, it operates simultaneously in time and frequency, making it suitable for series with local patterns at different resolutions.

```
# Apply Wavelet filter
filter <- ts_fil_wavelet()
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- The series is decomposed into **multiple scales** (resolutions)
- Detail components associated with noise are removed through thresholding
- The signal is reconstructed from the remaining components
- Suitable when the series contains local patterns at different resolutions

📌 **Key concept:** Wavelets operate in **time and frequency simultaneously** — unlike FFT, which is frequency only. Ref.: Donoho & Johnstone (1994).

EMD Filter – Empirical Mode Decomposition

EMD (Empirical Mode Decomposition) decomposes the series into Intrinsic Mode Functions (IMFs) extracted directly from the data, without a predefined fixed basis. High-frequency IMFs (noise) are discarded during reconstruction, preserving the non-linear and non-stationary structure of the signal.

```
# Apply EMD filter (reconstruction by IMFs)
filter <- ts_fil_emd()
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- EMD decomposes the series into **Intrinsic Mode Functions (IMFs)** extracted from the data
- Does not assume a fixed basis — completely **data-driven**
- High-frequency IMFs (noise) are discarded during reconstruction
- Preserves the non-linear and non-stationary structure of the signal

📌 **Key concept:** EMD is an **adaptive** decomposition — it does not use fixed bases like Fourier or Wavelets. Ref.: Huang et al. (1998).

REMD Filter – Robust EMD

REMD (Robust EMD) adds ensemble strategies — such as EEMD and CEEMDAN — to the standard EMD to stabilize the decomposition and reduce mode mixing, a common problem when the series contains strong noise or outliers. It is the preferred choice when standard EMD produces unstable IMFs.

```
# Apply REMD (Robust EMD)
filter <- ts_fil_remd()
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- REMD adds ensemble strategies (e.g., EEMD/CEEMDAN) to stabilize the decomposition
- Reduces **mode mixing** — a common problem in standard EMD under strong noise
- High-frequency IMFs are attenuated; remaining components are summed
- Use when the series is non-linear/non-stationary with intense noise and outliers

📌 **Key concept:** REMD is less sensitive to noise and outliers than standard EMD. Ref.: Souza et al., IJCNN 2024. doi:10.1109/IJCNN60899.2024.10651192

HP Filter – Hodrick-Prescott

The HP (Hodrick-Prescott) filter solves an optimization problem that balances data fidelity and trend smoothness, penalizing the sum of squared second differences. Widely used in econometrics to separate trend and cycle. Note: it may introduce bias at the endpoints of the series.

```
# Apply HP filter (trend extraction)
filter <- ts_fil_hp()
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- HP solves an optimization problem: data fidelity vs. trend smoothness
- Penalizes the sum of squared second differences of the trend
- Higher lambda → smoother trend and more aggressively filtered cycle
- **Note:** HP may introduce bias at the endpoints of the series

📌 **Key concept:** HP separates **trend and cycle** via penalized optimization. Widely used in economics. Ref.: Hodrick & Prescott (1997).

Kalman Filter – State Space

The Kalman filter models the series as a state space system with a latent state that is updated with each new observation. It explicitly separates observation noise (H) from process dynamics (Q), offering a fundamentally different perspective from window-based smoothing.

```
# Apply Kalman filter (H=obs. noise, Q=process noise)
filter <- ts_fil_kalman(H = 0.1, Q = 1)
filter <- fit(filter, tsd$y)
y <- transform(filter, tsd$y)

plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- H: observation noise variance; Q: process noise variance
- The filter maintains a **latent state** and updates it with each new observation
- Explicitly separates observation noise from the underlying state dynamics
- Small H → trusts observations more; large Q → state changes rapidly

📌 **Key concept:** Kalman filters via a **state space model** — a different perspective from window-based smoothing. Ref.: Kalman (1960).

Seasonal Adjustment Filter (STL)

The Seasonal Adjustment filter estimates and removes recurring seasonal effects from the series using STL decomposition (Seasonal-Trend decomposition via Loess). The resulting series reflects only non-seasonal dynamics (trend + irregular component), facilitating modeling and forecasting. Requires that the data periodicity is adequate for seasonal estimation.

```
# Apply seasonal adjustment (STL-based)
```

```
filter <- ts_fil_seas_adj()
```

```
filter <- fit(filter, tsd$y)
```

```
y <- transform(filter, tsd$y)
```

```
plot_ts_pred(y=tsd$y, yadj=y)
```

Step by Step

- Estimates and removes **recurring seasonal effects** from the series
- Approach based on STL (Seasonal-Trend decomposition via Loess)
- The resulting series reflects only non-seasonal dynamics (trend + irregular)
- Verify that the data periodicity is adequate for seasonal estimation

📌 **Key concept:** Seasonal adjustment removes **periodic patterns** before modeling, facilitating trend and cycle analysis. Ref.: Cleveland et al. (1990).

Recommended Workflow

Baseline

Apply `ts_fil_none()` and visualize the raw series.

Diagnosis

Identify the type of unwanted variation (noise, spikes, seasonality).

Selection

Choose the filter appropriate to the problem family.

Evaluation

Compare the filtered series with the baseline.

Following this workflow ensures that each filtering decision is justified by visual evidence and direct comparison with the unfiltered baseline.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

